University of Tlemcen Abou Bekr Belkaid
Sciences faculty
Computer Science Department
2025/2026

ING 3 IA --- Mobile Applications Practicum

Practical 5: REST API

1) **Introduction :** the objective of this practical is to learn how to fetch data from a REST API and integrate it into a Flutter project. The first step is to install the http package, which is required to make HTTP requests in Flutter. The exercise is then divided into two main steps: fetching data from the REST API and displaying the retrieved data in the user interface.

2) **Package installation :** to interact with a REST API, it is necessary to install and import the ***http*** (or ***dio***) package. This can be done by running *"flutter pub add http"* command followed by *"flutter pub get"*. Then, import the package in the Dart file using *"import 'package:http/http.dart'"*.

3) **Fetching the data:** fetching data from the REST API consists of defining the API URL, sending an HTTP GET request, and retrieving the server's response. Since, the response is returned in JSON format, it must be decoded using *jsonDecode* to transform it into a Dart structure (a Map).
   a. ***Defining the "fromJson" method:*** the decoded JSON data (Map) cannot be used directly in the application. It must first be converted into a *Meal* object that matches the model used in the project. For this reason, a ***"fromJson"*** method needs to be added in the ***"Meal.dart"*** file. This method takes a ***Map*** (representing the JSON object) and returns a ***Meal*** instance.

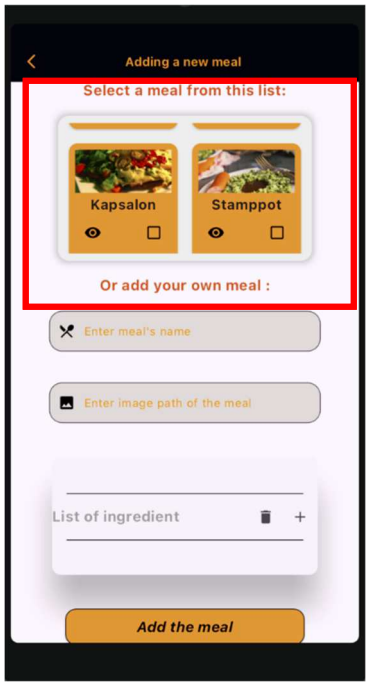
```dart
static Meal fromJson(Map<String, dynamic> element) {
       Meal mealTMP = Meal(
                      name: element['strMeal'],
                      imgPath: element['strMealThumb'],
                      listOfIngredients: [],
                      identifier: element['idMeal'],
                      );
      return mealTMP;
}
```

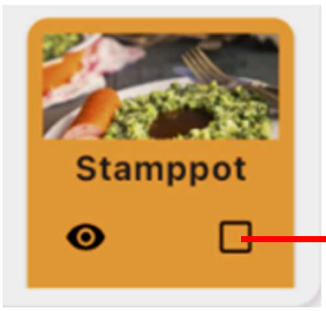
b. ***Fetching the meals list from the REST API server:*** in this practicum, the *"themealdb.com"* API is used as the data source that provides meal information through the following endpoint: https://www.themealdb.com/api/json/v1/1/search.php?s=p
To retrieve the meals from the REST API, the ***"FetchMealsFromAPI"*** is defined inside the ***"AddNewMealScreen"*** page. This function send an HTTP GET request to the endpoint and receives a JSON response. The returned JSON data is then decoded, and each item is converted into a ***Meal*** object using the ***fromJson*** constructor provided by the *"dart:convert"* package (*import 'dart:convert';* to use ***fromJson***).

```dart
Future<List<Meal>> fetchMealsFromAPI() async {
    List<Meal> mealListTMP = [];
    var fetchedData = await get(
    Uri.parse('https://www.themealdb.com/api/json/v1/1/search.php?s=p'));
       if (fetchedData != null) {
       //Decode the fetched data (JSON data → Dart MAP)
           var decodedData = jsonDecode(fetchedData.body);
           //Add each fetched meal to the list
               if (decodedData['meals'].isNotEmpty) {
                   for (final element in decodedData['meals']) {
                        mealListTMP.add(Meal.fromJson(element));
                   }
               }
       }
    return mealListTMP;
}
```

4) **Displaying Meals List in the UI:** this task consists of integrating the list of meals retrieved from the REST API into the application's UI.

   a. ***Redesigning the "AddNewMealScreen" page:*** as shown in Figure 1, a ***GridView.builder*** displaying the meals fetched from the REST API should be integrated to the existing UI of the ***"AddNewMealScreen"*** page. To achieve this, a ***"FutureBuilder"*** widget is used to call the ***"FetchMealsFromAPI"*** function and it waits for its result. Once the data is returned, the list of meals contained in *"snapshot.data"* is stored in a variable named "*fetchedMealList*". Inside the ***"FutureBuilder"***, a *"GridView.builder"* dynamically generates a grid of meal cards (i.e. each element of the fetched list is represented by a ***"MealCard"*** component). Finally, the ***onDeleteMeal*** method of the ***MealCard*** should be implemented such as the meal is selected as a new meal and returned to the ***"WeekDaysCard"*** page.






```
child: Column(
children: [
//// Suggest to select a meal from the API////
const Text("Select a meal from this list:",
style: TextStyle( fontSize: 20,color: Colors.deepOrange,
                  fontWeight: FontWeight.bold)),
Container( width: 350, height: 200,
           padding: const EdgeInsets.all(10),
           child: FutureBuilder(
                  future: fetchMealsFromAPI(),
                  builder: (context, snapshot) {
if (snapshot.hasData && snapshot.data == true) {
                  final fetchedMealList = snapshot.data!;
                  return GridView.builder(
                  itemCount: fetchedMealList.length,
                  gridDelegate:
           const SliverGridDelegateWithFixedCrossAxisCount(
                  crossAxisCount: 2,
                  crossAxisSpacing: 10,
                  mainAxisSpacing: 10,
                  childAspectRatio: 0.8,
                  ),
                  itemBuilder: (context, index) {
                        return MealCard(
                           meal: fetchedMealList[index],
                           onDeleteMeal: () {
 // … Here, when a meal is selected, it will be assigned to
     theNewMeal …//
                  theNewMeal = fetchedMealList[index];
// Send the new Meal to the WeekDaysCard
                   Navigator.of(context).pop(theNewMeal);
                              },
                           );
                        },
                     );}
               }),
          ),
const Text("Or add your own meal :",
style: TextStyle(fontSize: 20,color: Colors.deepOrange,)),
SizedBox(height: screenHeight * 0.05),
```

Figure 1: The new design of the *"AddNewMealScreen"* page

b. UI Adaptation (i.e. "MealCard" adjustment): the ***"MealCard"*** widget must now be adapted to handle the new situation where the meal image may come from two different sources: either a local image stored in the project, or an image provided by the REST API. To handle this, the "rootBundle.load" method is used to check whether the given image path refers to a local asset or an external URL. If the image exists locally, the card displays it with *Image.asset*, otherwise, it displays the image from the API using *Image.network*.

```
child: rootBundle.load(meal.imgPath)==true ?
               Image.asset( meal.imgPath, height: screenHeight * 0.1,
                            width: screenWidth * 0.25,):
               Image.network( meal.imgPath, height: screenHeight * 0.1,
                            width: screenWidth * 0.25,),
```

Finally, the icon displayed in the ***"MealCard"*** widget depends on the screen from which the widget is used. When the ***"MealCard"*** is used in the ***"MealsOfADayScreen"***, the icon should be a *"delete"* icon and when it is used in the ***"AddNewMealScreen"***, the icon should be a *"checkbox"* icon. To support this behavior, a new attribute named ***"cardIcon"*** should be added to the ***"MealCard"*** widget. This attribute allows each screen to specify the appropriate icon when creating a ***"MealCard"*** instance.

```
class MealCard extends StatelessWidget {
final Meal meal;
final Function() onDeleteMeal;
final Icon cardIcon;

const MealCard({super.key, required this.meal, required this
.onDeleteMeal, required this.cardIcon});
.
.
.
.

child: IconButton(
icon: cardIcon,
```

```
// From MealsOfADayScreen
return MealCard( cardIcon: Icon(Icons.delete),
… … … …
```

```
// From AddNewMealScreen
return MealCard( cardIcon:Icon(Icons.check_box_outline_blank),
… … … …
```